

TP2 – Arianagram: Documentação Técnica

Repositório: <https://github.com/joao-v-gomes/tp2-redes-arianagram>

O Arianagram é um servidor multithread de rede social sobre TCP, seguindo o modelo **Single-Process, Multi-Thread (SPMT)**: um único processo com um laço principal parado no `accept()`, e a cada conexão nova cria uma thread dedicada ao cliente.

1. Estruturas de Dados do Servidor

Todas as estruturas compartilhadas são globais e cada uma tem o seu próprio mutex. Usei arrays de tamanho fixo em vez de listas mais complexas.

1.2 Clientes conectados

Um array com os dados de uma conexão:

```
typedef struct {
    int active;
    int client_fd;
    pthread_t id;
    char username[USER_SIZE];
    pthread_mutex_t send_mutex; // protege escritas no socket deste
    cliente
} client_t;

client_t clients[MAX_CLIENTS]; // MAX_CLIENTS = 128
pthread_mutex_t clients_mutex; // protege a busca/ocupação de slots
```

O `clients_mutex` protege a varredura e a inserção de novas conexões. O `send_mutex` é utilizado para fazer o envio dos dados pelo cliente.

1.3 Feed global

Um array circular de 5 posições. Os valores que chegam sempre ocupam a primeira posição. Mais novos no começo e mais antigos no final.

```
Message feed_messages[FEED_MAX_SIZE]; // FEED_MAX_SIZE = 5
pthread_mutex_t feed_mutex;

...
for (int i = FEED_MAX_SIZE - 1; i > 0; i--)
    feed_messages[i] = feed_messages[i - 1];
feed_messages[0] = *msg;
```

Usei `msg_id != 0` pra saber se uma posição do feed é válida (como os IDs começam em 1, zero significa vazio).

1.4 Seguidores

Foi a única estrutura que reescrevi do zero no meio do caminho.

A primeira versão era uma **matriz indexada por slot de cliente**:

```
// followers[i][j] = 1 -> o slot i é seguido pelo slot j
int followers[MAX_CLIENTS][MAX_CLIENTS] = {0};
```

Cada linha era um usuário e cada coluna era um seguidor mas, testando, percebi que indexar por *slot* quebrava em três situações que o próprio enunciado manda tratar, principalmente depois da mudança no enunciado:

- **Seguir alguém que ainda não conectou:** se o alvo não tinha slot, não tem como marcar a relação. O enunciado diz que um FOLLOW de usuário inexistente tem que ser registrado mesmo assim, pra ser entregue se ele aparecer depois.
- **Reuso de slot:** quando um cliente desconecta, o slot dele é liberado e pode ser usado por outro cliente. As marcações da matriz naquela linha/coluna passavam a valer pro cliente novo, isto é, ele recebia automaticamente os follows de quem ocupava o slot antes.
- **Nome de usuário duplicado:** dois clientes com o mesmo nome ficam em slots diferentes. Como o push tem que ir pra *todos* os sockets daquele nome, amarrar a relação a um slot só não batia com a semântica de "mesmo perfil".

Por causa disso troquei pela lista de pares por nome de usuário. Ficou mais complicado mas resolveu melhor o meu problema.

```
typedef struct {
    char follower[USER_SIZE]; // quem segue
    char followed[USER_SIZE]; // quem é seguido
} follow_t;

follow_t follow_list[MAX_FOLLOWS];
int follow_count;
pthread_mutex_t follow_mutex;
```

A relação passou a ser por nome, independente do estado de conexão, e os três problemas somem de uma vez: consigo seguir quem ainda não se conectou (guardo os nomes e entrego quando ele postar), o reuso do slot deixa de importar (follow automático), e um FOLLOW de um usuário passa a valer pra qualquer socket ativo com esse nome de usuário.

1.5 Contador de ID

Um `uint32_t` global com o seu mutex, pra garantir IDs sequenciais e únicos mesmo com várias threads postando ao mesmo tempo.

2. Ciclo de Vida de uma Thread

Cada cliente é atendido por uma thread que nasce, processa comandos e morre.

2.1 Criação

O loop principal aceita a conexão, procura um slot em `clients[]` usando o `clients_mutex`, inicializa o slot e cria a thread passando o índice dele:

```
pthread_create(&id, NULL, handleClientConnection, (void *)
(intptr_t)client_index);
```

2.2 Operação

A primeira coisa que a thread faz é se desanexar, pra liberar os recursos automaticamente sem ninguém precisar dar `join` nela:

```
pthread_detach(pthread_self());
```

Depois entra num loop de uma FSM, mesma ideia do TP1, lendo o socket: `WAITING_FOR_MESSAGE` → `RECEIVED_MSG` → seleção pelo tipo da mensagem (CONNECT, POST, FOLLOW, READ). A leitura usa um loop de `recv` até completar a `Message` de tamanho fixo. No POST, a própria thread também escreve nos sockets de outros clientes.

2.3 Encerramento:

Quando o `recv()` retorna 0 (FIN do cliente) ou um valor negativo (erro) significa que a conexão caiu. A thread marca o slot como inativo, loga a saída e fecha o socket. Pego o username do socket antes de fechar o slot para usar no `[DISC]`.

```
clients[client_index].active = 0;
printf("[DISC] %s desconectou.\n", disc_username);
close(client_fd);
return NULL;           // como deu detach, os recursos são liberados aqui
```

3. Desafios do Push para Sockets de Outros Clientes

Esse foi o ponto mais delicado do trabalho. Diferente de um servidor de requisição-resposta, aqui a thread do poster escreve no socket de outro cliente. Dois escritores podem cair no mesmo socket ao mesmo tempo, por exemplo, um push chegando enquanto a thread daquele cliente está enviando o feed de um `READ`. Como as mensagens têm tamanho fixo e mando com `send` direto, sem proteção os bytes de duas `Message` se misturam e o cliente recebe lixo.

A solução foi um **mutex de escrita por cliente** (`send_mutex`). Toda escrita no socket, tanto o PUSH quanto o feed inteiro do READ, aciona esse lock:

```
pthread_mutex_lock(&clients[idx].send_mutex);
if (clients[idx].active && strcmp(clients[idx].username, to_notify[f]) ==
0)
    send(clients[idx].client_fd, &push_msg, sizeof(Message), 0);
pthread_mutex_unlock(&clients[idx].send_mutex);
```

Como o slot pode ser usado por outro cliente entre o momento em que listo os seguidores e o momento do envio, eu verifico o `active` e o nome novamente.

Outro ponto importante foi a **ordem de aquisição dos locks**, pra não dar deadlock. A regra que segui foi nunca pegar dois locks ao mesmo tempo: copio a lista de seguidores usando o `follow_mutex` e libero; pego o `clients_mutex`, uso e libero, e só depois mando a mensagem usando o `send_mutex`. O feed global tem o mesmo tratamento usando o `feed_mutex`, para somente um socket escrever no array, seguindo o enunciado do TP.

O caso do **nome duplicado**: como a relação é por nome, ao notificar um seguidor eu leio o `clients[]` e mando o push pra todos os sockets ativos com aquele nome, não só pro primeiro.

4. Decisões de Projeto e Justificativas

- **SPMT (uma thread por cliente)**: além de ser o que o enunciado pede, isola cada sessão. Uma conexão lenta ou que caiu não trava as outras.
- **Mensagens de tamanho fixo + big-endian**: converto o `type` com `htons/ntohs` e o `msg_id` com `htonl/ntohl`. Sem isso, um cliente compilado em outra máquina poderia ler o tipo e o ID errados; é o que garante a interoperabilidade com o outros cliente. Isso foi alertado pelo monitor nos comentarios do TP1.
- **Seguidores por nome, não por slot**: Usar o nome do usuário foi muito mais fácil e seguro do que por slot no array e resolveu os problemas de usuário inexistente e nome duplicado.
- **Vários locks em vez de um só**: separei `clients_mutex`, `feed_mutex`, `follow_mutex`, `msg_id_mutex` e o `send_mutex` por cliente. Um lock global único seria mais simples, mas serializaria tudo; com locks separados, duas threads podem, por exemplo, mandar push em sockets diferentes ao mesmo tempo. Usando somente um `mutex`, eu teria que esperar um cliente realizar uma ação para poder realizar outra de outro cliente.
- **pthread_detach em vez de join**: a main nunca precisa esperar uma thread de cliente terminar — ela só aceita conexões. Desanexar deixa a limpeza automática e simples. Nos testes iniciais usando `pthread` tive problema com isso e o `detach` resolveu.